GraphWord

Rivero Sánchez, Raúl — Medina Sosa, Pablo

University of Las Palmas de Gran Canaria, Las Palmas de Gran Canaria, Spain.

Repositorio de Github

1 Project Introduction

The GraphWord project aims to implement a scalable and distributed architecture on AWS to manage graphs that represent relationships between words. This system leverages services in the cloud, DevOps practices, and design principles geared toward resilience and performance. An API was built that allows advanced topological operations to be performed on graphs, such as finding minimum paths, identifying clusters, or determining isolated nodes, among others.

2 Architecture Overview

The architecture implemented for this project has been designed with the aim of optimizing scalability, security and efficiency in graph management and analysis. It is supported by a combination of managed services and custom configurations. AWS solutions that meet the technical and functional requirements of the system. The modular design of the architecture also facilitates future extensions and integration with other systems. Its main components and their interaction are described below:

The project follows a strategy of design based on distributed and decentralized services to maximize availability and performance. In particular, the separation between public and private subnets is key to divide responsibilities and ensure a higher level of security. This separation allows resources with different levels of sensitivity to operate in a controlled and secure environment, while taking advantage of efficient connectivity between different services.

In addition, the architecture leverages native AWS services to reduce operational complexity. For example, Lambda functions eliminate the need to manage traditional servers and allow automatic scaling based on load. Similarly, S3 buckets act as persistent and highly available data stores, simplifying data management and access from other parts of the system.

Regarding access to external resources, the use of the NAT Gateway and the Gateway Endpoint for S3 ensures granular control over outbound connections and interactions with external services. This configuration not only reduces transfer costs, but also increases security by keeping traffic private within the AWS infrastructure. The architecture implemented for this The project has been designed with the aim of optimizing scalability, security and efficiency in graph management and analysis. It is supported by a combination of managed services and custom AWS configurations that allow the technical and functional requirements of the system to be met.

The following figure shows a business and infrastructure diagram that represents the architecture we have implemented for this project:

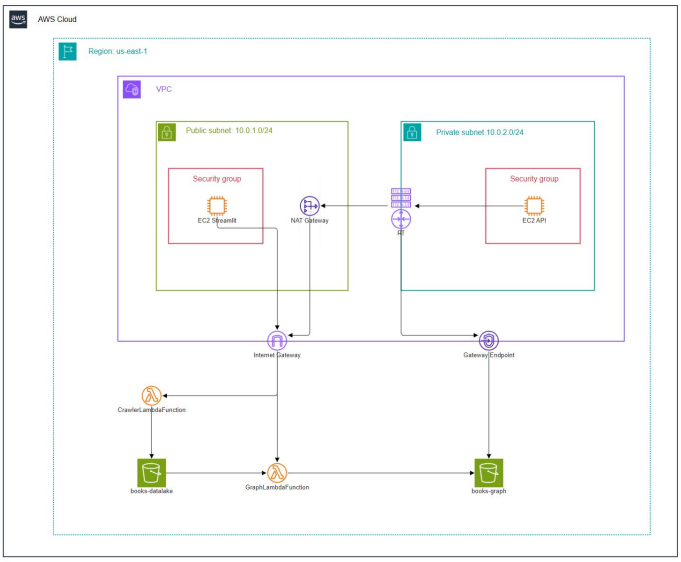

Figure 1: Business and Infrastructure Diagram

The main components and their interaction are described below:The architecture is implemented in AWS and follows a distributed and scalable structure consisting of the following main elements: beginenumerate

VPC (Virtual Private Cloud):

- Isolated logical space that houses all project resources.
- Configured with two subnets:
  - textbfPublic Subnet (10.0.0.1/24): Hosts services that require direct internet access. This IP range was selected to allow for better management of publicly exposed resources, as it provides adequate IP space for the intended public instances and services, and simplifies the configuration of access and security rules.
  - **Private Subnet** (10.0.2.0/24): Hosts critical services that do not require direct internet access, improving security. This IP range contributes to the isolation of theinternal resources by logically separating sensitive system components. It offers advantages such as protection against unauthorized access from the Internet and greater ease in enforcing strict security policies. In addition, this range was selected to avoid conflicts with other networks in the organization and ensure compatibility with future expansions.

EC2 Instances:

- **Streamlit Server (Public Subnet)**:
  - Implements the graphical interface of the system, accessible from the outside.
  - Configured to communicate with the internal services of the graph.
- **API Server (Private Subnet)**:
  - Provides the REST API endpoints to interact with the graphs.
  - Protected behind a gateway to restrict direct access from the internet.

Lambdas:

- **CrawlerLambdaFunction**: Responsible for downloading books from external sources and storing them in an S3 bucket called `books-datalake`.
- **GraphLambdaFunction**: Generates graphs from the processed books and stores them in an S3 bucket called `books-graph`.

S3 Buckets:

- **books-datalake**: Initial repository of unprocessed data.
- **books-graph**: Store of the generated graphs in JSON format.

Internet Gateway and Gateway Endpoint:

- The **Internet Gateway** enables communication between resources in the public subnet and the internet.
- The **Gateway Endpoint** facilitates secure communication between resources in the private subnet and S3 buckets.

Security:

- Use of strict **security groups** that control incoming and outgoing traffic.
- EC2 instances and Lambda functions have specific IAM roles and policies to ensure controlled access to resources.

3 Design Decisions

1. **Separation of Subnets**: The choice to separate services into public and private subnets was key to improving security and efficiency. Sensitive resources, such as the API Server and S3 buckets, are protected in the private subnet.
2. **Use of Lambdas**: Lambda functions were selected for their scalability and ability to run on demand, optimizing costs and simplifying infrastructure management.
3. **Persistence on S3**: S3 was used as a scalable and cost-effective storage solution for processed and unprocessed data.
4. **DevOps and CI/CD**: Continuous integration and delivery were implemented to ensure an automated and reproducible deployment of the system.

3.1 API Operations

The REST API allows for multiple analyses to be performed on graphs, including:

- Finding shortest paths between words.
- Identifying isolated or highly connected nodes.
- Detecting clusters within the graph.
- The API also has the ability to trigger Lambda functions (CrawlerLambdaFunction and GraphLambdaFunction) on a scheduled or as-needed basis, extending its functionality to cover dynamic processing tasks.

Each operation is associated with a specific endpoint, protected under the private subnet security settings.

3.2 Scalability and Future

The current design is prepared to handle increasing volumes of data thanks to the use of AWS Lambda and S3. However, future iterations could consider:

1. **Using specialized databases**: Incorporating Neo4j to manage advanced queries on graphs.
2. **Optimizing graph generation algorithms**: Improving performance using parallelization techniques or frameworks such as Apache Spark.

4 More Economically Possible Improvements

With a larger budget, the following improvements to the architecture could be implemented:

4.1 High Availability

- Deploying EC2 instances in multiple availability zones to ensure service continuity in the event of regional failures.
- Configuring load balancers (Elastic Load Balancers) to distribute traffic between instances and improve performance.

4.2 Managed Databases

- Replace the databases S3 buckets as primary storage with a managed database system such as Amazon Aurora or DynamoDB to optimize queries and data storage.

4.3 Advanced Security

- Deploy AWS WAF (Web Application Firewall) to protect web applications from common threats.
- Use AWS GuardDuty for automated detection of threats and potential vulnerabilities.

4.4 Cost and Performance Optimization

- Migrate Lambda functions to Fargate or Managed Kubernetes (EKS) for tasks with constant processing requirements.
- Incorporate Reserved or Cost-Saving Instances for predictable services.

4.5 Automation and Monitoring

- Incorporate tools such as AWS CloudFormation to automate infrastructure deployment.
- Implement Amazon CloudWatch Logs Insights for real-time monitoring and anomaly detection.

4.6 Artificial Intelligence and Machine Learning

- Integrate Machine Learning services such as Amazon SageMaker for advanced graph analysis and pattern-based predictions.

These improvements would not only increase system capacity and resilience, but would also offer a more advanced and fluid user experience.